ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT - HÀN
[Chưa cung cấp tên Khoa]

BÁO CÁO ĐỒ ÁN HỌC PHẦN
LẬP TRÌNH HỆ THỐNG (SYSTEM PROGRAMMING)

ĐỀ TÀI 17:
CÀI ĐẶT LỆNH LS VỚI CÁC TÙY CHỌN -A, -L, -R, -S TRÊN LINUX
(Implement ls with -a, -l, -R, -S)

Giảng viên hướng dẫn:	[Chưa cung cấp Giảng viên hướng dẫn]
Nhóm thực hiện:	[Chưa cung cấp danh sách nhóm]
Học kỳ / Năm học:	[Chưa cung cấp Học kỳ / Năm học]
Thành viên thực hiện:	[Chưa cung cấp danh sách thành viên nhóm (Họ tên, MSSV, Lớp)]

Lưu ý bản nháp (Draft): Thông tin danh sách nhóm, MSSV, Lớp, Khoa và GVHD được trích xuất từ tệp cấu hình docs/project_metadata.json và cần cập nhật trước khi xuất bản nộp cuối.

Đà Nẵng – Năm 2026

MỤC LỤC

MỞ ĐẦU 1
CHƯƠNG 1. PHÂN TÍCH YÊU CẦU VÀ HỢP ĐỒNG HÀNH VI 2
1.1. Yêu cầu chức năng của bốn tùy chọn (-a, -l, -R, -S) 2
1.2. Hợp đồng hành vi cho tổ hợp cờ, đối số hỗn hợp và mã thoát 2
1.3. Ranh giới phạm vi và các tính năng ngoài phạm vi đề tài 3
CHƯƠNG 2. CƠ SỞ KỸ THUẬT HỆ THỐNG TỆP LINUX VÀ GIAO DIỆN POSIX 4
2.1. Cấu trúc danh mục thư mục (Directory Entry) và siêu dữ liệu (Inode) 4
2.2. Liên kết cứng (Hard link), Liên kết mềm (Symlink) và lstat() vs stat() 4
2.3. Giải mã trường st_mode và số hiệu thiết bị major/minor 5
2.4. Tổng hợp các giao diện hệ thống và thư viện C chuẩn sử dụng 6
CHƯƠNG 3. THIẾT KẾ KIẾN TRÚC VÀ CÀI ĐẶT CHƯƠNG TRÌNH 8
3.1. Kiến trúc phân tách 5 module và luồng dữ liệu tổng thể 8
3.2. Cấu trúc dữ liệu cốt lõi và quản lý quyền sở hữu bộ nhớ (Ownership) 9
3.3. Thuật toán duyệt đệ quy (-R), chốt chặn vòng lặp và đánh đổi tài nguyên 10
3.4. Thuật toán sắp xếp (-S) và định dạng kết xuất hai lượt (-l) 11
CHƯƠNG 4. KIỂM THỬ THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ 12
4.1. Môi trường thực nghiệm và thiết kế cây dữ liệu kiểm thử (Fixtures) 12
4.2. Phương pháp kiểm thử và độ bao phủ kịch bản 12
4.3. Minh chứng đầu ra thực nghiệm từ log kiểm thử và demo 13
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 15
5.1. Kết quả đạt được 15
5.2. Hạn chế hiện tại và hướng phát triển 15
TÀI LIỆU THAM KHẢO i
PHỤ LỤC. DANH SÁCH KỊCH BẢN KIỂM THỬ TỰ ĐỘNG VÀ LỆNH TÁI LẬP ii
Phụ lục A. Quy trình lệnh kiểm chứng độc lập trên Ubuntu WSL2 ii
Phụ lục B. Danh sách chi tiết 37 kịch bản trong bộ kiểm thử tự động ii

DANH MỤC TỪ VIẾT TẮT VÀ THUẬT NGỮ

Bảng 0.1: Giải thích các từ viết tắt và thuật ngữ sử dụng trong báo cáo

Từ viết tắt	Thuật ngữ đầy đủ	Ý nghĩa kỹ thuật trong đồ án
ASan / LSan / UBSan	AddressSanitizer / LeakSanitizer / UndefinedBehaviorSanitizer	Bộ công cụ phát hiện lỗi bộ nhớ và hành vi bất định của trình biên dịch GCC/Clang.
CLI	Command-Line Interface	Giao diện dòng lệnh nhận cờ tùy chọn và toán hạng đường dẫn qua argc, argv.
Directory entry / dentry	Mục thư mục / bộ đệm tên VFS	readdir trả struct dirent ở user space; dentry là cấu trúc bộ đệm đường dẫn trong kernel, không đồng nhất hai cấu trúc.
DrvFs	Drive File System	Trình điều khiển gắn kết phân vùng Windows NTFS vào WSL2 (/mnt/c, /mnt/p).
EOF	End Of File	Trạng thái kết thúc luồng đọc dữ liệu hoặc kết thúc danh mục thư mục.
FD	File Descriptor	Số nguyên không âm đại diện cho tài nguyên tệp hoặc thư mục đang mở của tiến trình.
GID / UID	Group ID / User ID	Định danh số của nhóm sở hữu và người dùng sở hữu đối tượng tệp.
Inode	Index Node	Cấu trúc lưu trữ siêu dữ liệu của tệp trên hệ thống tệp Unix/Linux.
POSIX	Portable Operating System Interface	Bộ tiêu chuẩn IEEE 1003.1 quy định giao diện lập trình hệ điều hành.
SUID / SGID	Set User ID / Set Group ID	Bit đặc biệt của tệp thực thi; SGID trên thư mục còn liên quan kế thừa nhóm. Cách áp dụng phụ thuộc hệ thống tệp và chính sách mount.
VFS	Virtual File System	Lớp trừu tượng hệ thống tệp ảo bên trong nhân Linux.
WSL2	Windows Subsystem for Linux 2	Môi trường chạy nhân Linux thực trên nền tảng ảo hóa nhẹ của Windows.

DANH MỤC HÌNH VẼ VÀ BẢNG BIỂU

Danh mục hình vẽ

Hình 2.1: Mối quan hệ giữa bảng danh mục thư mục (readdir), Inode (lstat/stat), Hard link và Symbolic link 5

Hình 3.1: Sơ đồ luồng thực thi tổng thể từ phân tích tham số dòng lệnh đến kết xuất và đệ quy 8

Hình 3.2: Thứ tự duyệt -aR và quy tắc bỏ qua . / .. / symlink khi đi xuống cây 10

Hình 3.3: Giải phẫu các trường thông tin trên một dòng kết xuất ở chế độ -l 11

Hình 4.1: Đầu ra thực nghiệm trình bày lại từ log kiểm thử tự động (37/37 kịch bản đạt) 13

Hình 4.2: Đầu ra thực nghiệm trình bày lại từ log demo cho các tùy chọn mặc định, -a và -lS 14

Hình 4.3: Đầu ra thực nghiệm trình bày lại từ log demo cho tổ hợp -alRS và xử lý đối số lỗi 14

Danh mục bảng biểu

Bảng 0.1: Giải thích các từ viết tắt và thuật ngữ sử dụng trong báo cáo ii

Bảng 1.1: Đặc tả chức năng và ràng buộc kỹ thuật của bốn tùy chọn trong Đề tài 17 2

Bảng 1.2: Hợp đồng hành vi chi tiết của chương trình myls 3

Bảng 2.1: Cấu trúc các vùng bit trong trường st_mode và quy tắc chuyển đổi sang chuỗi 10 ký tự 5

Bảng 2.2: Danh mục các API chuẩn POSIX và C11 sử dụng trong chương trình myls 6

Bảng 3.1: Phân công trách nhiệm giữa các tệp mã nguồn trong dự án 8

Bảng 4.1: Tổng hợp kết quả thực nghiệm từ tệp dữ liệu evidence/test_results.json 13

Bảng P.1: Danh sách toàn bộ 37 kịch bản kiểm thử tự động và trạng thái thực thi ii

MỞ ĐẦU

1. Đặt vấn đề và lý do chọn đề tài: Trong các hệ điều hành họ Unix và Linux, tiện ích dòng lệnh ls là công cụ nền tảng để quan sát không gian tên thư mục và kiểm tra siêu dữ liệu của các đối tượng tệp [1], [2]. Tuy nhiên, đối với học phần Lập trình Hệ thống, việc gọi hàm thư viện cấp cao như system("ls") chỉ khởi tạo một tiến trình vỏ (shell) chạy tệp nhị phân có sẵn của hệ điều hành mà không giúp người lập trình tiếp cận cơ chế quản lý danh mục thư mục, cấu trúc siêu dữ liệu Inode, quản lý bộ nhớ động hay kiểm soát File Descriptor.

2. Mục tiêu của đồ án: Đề tài 17 đặt mục tiêu nghiên cứu cơ chế làm việc của hệ thống tệp Linux thông qua giao diện chuẩn POSIX.1-2008 [1] và tự cài đặt chương trình dòng lệnh myls bằng ngôn ngữ C (chuẩn C11) hỗ trợ bốn tùy chọn cốt lõi: -a (hiển thị mục ẩn), -l (định dạng chi tiết), -R (duyệt đệ quy cây thư mục con) và -S (sắp xếp theo kích thước tệp giảm dần), cả khi dùng độc lập lẫn khi kết hợp tự do.

3. Phạm vi và phương pháp đánh giá: Chương trình được giới hạn rõ trong phạm vi một công cụ dòng lệnh chuẩn POSIX: sử dụng trực tiếp các hàm hệ thống và thư viện C chuẩn (opendir, readdir, closedir, lstat, stat, readlink, getpwuid, getgrgid, localtime_r, getopt, qsort) [3], [4]; không mở rộng sang giao diện đồ họa, không phụ thuộc vào các tiện ích ngoài và xác định rõ ranh giới những tính năng đặc thù của GNU ls không nằm trong phạm vi đề tài.

4. Thành viên và phân công: Thông tin nhóm chưa được cung cấp. Cần bổ sung đầy đủ họ tên, MSSV, lớp và phân công của tất cả thành viên trước khi nộp; bản thảo này không giả định số lượng hoặc danh tính thành viên.

CHƯƠNG 1. PHÂN TÍCH YÊU CẦU VÀ HỢP ĐỒNG HÀNH VI

1.1. Yêu cầu chức năng của bốn tùy chọn (-a, -l, -R, -S)

Chương trình myls nhận các tùy chọn dòng lệnh và không hoặc nhiều toán hạng đường dẫn (path operands) theo cú pháp chuẩn POSIX [1]: ./myls [-alRS] [file...]. Khi người dùng không truyền đường dẫn nào, chương trình mặc định liệt kê thư mục làm việc hiện tại ("."). Bảng 1.1 tổng hợp yêu cầu chức năng và ràng buộc kỹ thuật của từng tùy chọn.

Bảng 1.1: Đặc tả chức năng và ràng buộc kỹ thuật của bốn tùy chọn trong Đề tài 17

Tùy chọn	Hành vi hiển thị đầu ra	Ràng buộc kỹ thuật bắt buộc
Mặc định (không cờ)	Liệt kê các mục không bắt đầu bằng dấu '.'; in một tên mục trên mỗi dòng.	Sắp xếp tăng dần theo thứ tự byte của tên mục bằng hàm strcmp() (tương đương môi trường LC_ALL=C) [1], [2].
-a (All entries)	Hiển thị tất cả các mục trong thư mục, bao gồm tệp ẩn, thư mục ẩn và hai mục đặc biệt '.' và '..'.	Tắt bộ lọc kiểm tra ký tự đầu d_name[0] == '.' trong vòng lặp đọc thư mục.
-l (Long listing)	In dòng 'total <1KiB-blocks>' ở đầu mỗi thư mục; mỗi mục in: chuỗi 10 ký tự loại tệp và quyền, số hard link, tên owner, tên group, kích thước (hoặc major, minor), thời gian sửa đổi và đích symlink.	Cần thu thập toàn bộ mục trước khi in để tính tổng khối và tìm độ rộng lớn nhất của từng cột nhằm căn lề thẳng hàng.
-R (Recursive)	Sau khi hiển thị toàn bộ mục của thư mục hiện tại, tiếp tục duyệt đệ quy xuống các thư mục con thực sự theo đúng thứ tự đã sắp xếp.	Bắt buộc loại trừ '.' và '..' và không đi vào các symbolic link gặp trong cây thư mục để tránh vòng lặp vô tận [1].
-S (Sort by size)	Sắp xếp các mục theo kích thước st_size giảm dần (tệp lớn nhất đứng trước).	So sánh kiểu off_t (64 bit trên môi trường x86_64 đã thử) bằng toán tử quan hệ để tránh tràn số; nếu hai mục có cùng st_size thì phân định thứ tự theo tên tăng dần.

1.2. Hợp đồng hành vi cho tổ hợp cờ, đối số hỗn hợp và mã thoát

Để chương trình hoạt động nhất quán khi người dùng kết hợp nhiều tùy chọn (như -la, -lS, -aR, -alRS hoặc viết rời -S -R -l -a) và truyền nhiều đường dẫn hỗn hợp, hợp đồng hành vi của myls được xác định chặt chẽ qua các quy tắc trong Bảng 1.2.

Bảng 1.2: Hợp đồng hành vi chi tiết của chương trình myls

Khía cạnh hành vi	Quy tắc xử lý đã chốt trong chương trình
Đa toán hạng hỗn hợp (tệp + thư mục)	Tách thành hai danh sách: nhóm tệp đơn lẻ được sắp xếp và in trước (không có dòng total); nhóm thư mục được sắp xếp và duyệt sau, ngăn cách bằng một dòng trống.
Tiêu đề tên thư mục (<path>:)	Chỉ in dòng tiêu đề '<path>:' khi bật cờ -R hoặc khi tổng số toán hạng truyền trên dòng lệnh lớn hơn 1.
Toán hạng CLI là Symbolic link	Nếu không bật -l (hoặc đường dẫn kết thúc bằng dấu '/'), kiểm tra đích bằng stat(): nếu đích là thư mục thì liệt kê nội dung thư mục đích. Nếu bật -l và không có dấu '/' cuối, hiển thị bản thân symlink [1], [2].
Dấu phân cách '--' và '--help'	Dấu '--' kết thúc phần phân tích cờ để hỗ trợ tên tệp bắt đầu bằng dấu '-'; cờ '--help' in hướng dẫn sử dụng ra stdout và thoát với mã 0.
Mã thoát 0 (EXIT_LS_OK)	Toàn bộ đối số và thư mục được xử lý thành công, không phát sinh lỗi hệ thống hay lỗi ghi đầu ra.
Mã thoát 1 (EXIT_LS_MINOR_ERR)	Xảy ra lỗi cục bộ trên một mục hoặc một thư mục con trong quá trình duyệt cây (ví dụ thư mục con trong -R bị khóa quyền đọc chmod 000, hoặc lỗi readdir/readlink/closedir). Chương trình báo lỗi ra stderr và tiếp tục phần còn lại.
Mã thoát 2 (EXIT_LS_SERIOUS_ERR)	Toán hạng truyền trực tiếp trên CLI không tồn tại hoặc không mở được, tùy chọn dòng lệnh sai, hết bộ nhớ cấp phát động, hoặc lỗi ghi luồng stdout (ví dụ đầy thiết bị /dev/full) [2].
Tổng hợp nhiều lỗi	Sử dụng quy tắc giữ mức lỗi cao nhất (elevate_status): lỗi mức 1 xảy ra sau không bao giờ làm hạ mức lỗi 2 đã ghi nhận trước đó.

1.3. Ranh giới phạm vi và các tính năng ngoài phạm vi đề tài

Chương trình tập trung giải quyết trọn vẹn bốn tùy chọn của Đề tài 17 theo chuẩn POSIX. Các tính năng mở rộng đặc thù của GNU coreutils không thuộc phạm vi cài đặt và được ghi nhận tường minh gồm: (1) chế độ dàn nhiều cột theo độ rộng terminal (ioctl TIOCGWINSZ) và tô màu ANSI; (2) sắp xếp theo quy tắc ngôn ngữ bản địa (locale collation qua strcoll); (3) hiển thị danh sách kiểm soát truy cập mở rộng ACL (+) hoặc ngữ cảnh bảo mật SELinux (.); (4) tự động phát hiện chu trình gắn kết hệ thống tệp (mount point cycles) hoặc đồng bộ hóa khi cây thư mục bị biến đổi đồng thời bởi tiến trình khác trong lúc đang duyệt.

CHƯƠNG 2. CƠ SỞ KỸ THUẬT HỆ THỐNG TỆP LINUX VÀ GIAO DIỆN POSIX

2.1. Cấu trúc danh mục thư mục (Directory Entry) và siêu dữ liệu (Inode)

Trong kiến trúc Hệ thống tệp ảo (Virtual File System – VFS) của nhân Linux [4], tên tệp và siêu dữ liệu của tệp được lưu trữ ở hai cấu trúc tách biệt:

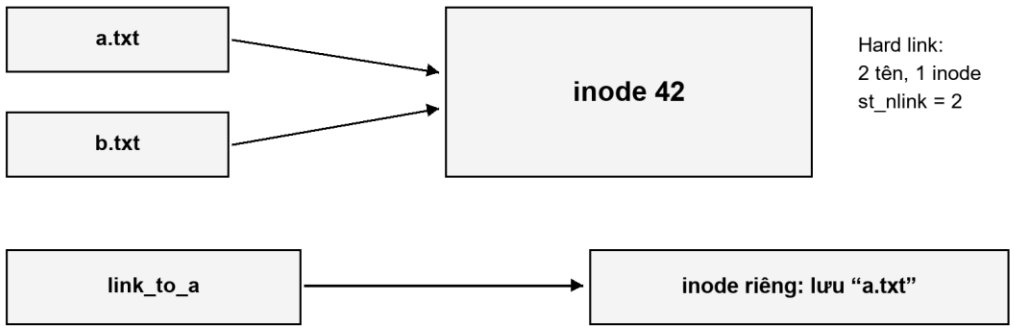
- **Mục danh bạ thư mục (Directory Entry):** Trên đĩa, một thư mục là một bảng ánh xạ từ tên mục (d_name) sang số hiệu định danh Inode (d_ino). Trong nhân Linux, các ánh xạ đường dẫn được lưu đệm tại dentry cache. Khi chương trình ở không gian người dùng gọi hàm thư viện readdir(3) [3], dữ liệu trả về là cấu trúc struct dirent chứa trường tên d_name và số d_ino. Cấu trúc struct dirent không chứa kích thước tệp, quyền truy cập, chủ sở hữu hay dấu thời gian sửa đổi.
- **Nút chỉ mục siêu dữ liệu (Inode và struct stat):** Mọi thuộc tính vật lý của tệp (loại tệp, 12 bit quyền truy cập, số lượng hard link st_nlink, định danh chủ sở hữu st_uid, định danh nhóm st_gid, kích thước byte st_size, số hiệu thiết bị st_rdev, số khối 512-byte st_blocks và thời gian sửa đổi st_mtime) đều nằm trong Inode của hệ thống tệp. Để đọc các thuộc tính này vào không gian người dùng, chương trình phải gọi lời gọi hệ thống lstat(2) hoặc stat(2) để nhận cấu trúc struct stat [3], [4].

Trong thiết kế này, de->d_name là tên cục bộ bên trong thư mục đang mở. lstat() giải đường dẫn tương đối theo thư mục làm việc của tiến trình, không theo DIR*. Vì chương trình không chdir(), join_path() ghép thư mục cha với tên mục (ví dụ demo_tree/src_module/main.c). Một thiết kế khác có thể dùng fstatat() với descriptor thư mục cha.

2.2. Liên kết cứng (Hard link), Liên kết mềm (Symlink) và lstat() vs stat()

Hệ thống tệp POSIX hỗ trợ hai cơ chế liên kết có bản chất hoàn toàn khác nhau, được minh họa trực quan tại Hình 2.1:

- **Liên kết cứng (Hard link):** Nhiều mục tên trong thư mục cùng ánh xạ trực tiếp tới một số hiệu Inode duy nhất trên cùng phân vùng. Khi tạo thêm một hard link, trường st_nlink trong Inode tăng thêm 1; dữ liệu chỉ bị xóa khỏi đĩa khi st_nlink giảm về 0 và không còn tiến trình giữ tệp mở.
- **Liên kết mềm (Symbolic link – Symlink):** Là một tệp đặc biệt có số hiệu Inode riêng và kiểu tệp S_IFLNK. Nội dung dữ liệu của symlink là một chuỗi ký tự biểu diễn đường dẫn tới tệp hoặc thư mục đích. Để lấy nội dung chuỗi đích này nhằm hiển thị hậu tố " -> target" trong chế độ -l, chương trình phải gọi hàm readlink(2) [3].



Hình 2.1: Mối quan hệ giữa bảng danh mục thư mục (readdir), Inode (lstat/stat), Hard link và Symbolic link

Như thể hiện ở Hình 2.1, việc lựa chọn giữa lstat() và stat() quyết định tính đúng đắn của chương trình: hàm stat() tự động đi xuyên qua symlink để đọc Inode của mục đích; nếu đích không tồn tại (broken symlink), stat() trả lỗi ENOENT khiến chương trình bỏ sót symlink đó, và nếu symlink trỏ ngược lên thư mục cha (ví dụ loop_parent -> ..), việc dùng stat() khi duyệt đệ quy -R sẽ nhận nhầm symlink là thư mục thật và gây lặp vô tận. Do đó, myls sử dụng lstat() cho toàn bộ các mục đọc từ thư mục, và chỉ sử dụng stat() để kiểm tra đích của toán hạng symlink được người dùng truyền trực tiếp trên dòng lệnh khi không bật -l hoặc khi có dấu '/' ở cuối đường dẫn [1], [2].

2.3. Giải mã trường st_mode và số hiệu thiết bị major/minor

Trường st_mode trong struct stat có kiểu mode_t. Trên Linux, các mask truyền thống ở 16 bit thấp biểu diễn loại tệp và quyền truy cập. Mã nguồn dùng S_IS* và các mask đặt tên, không giả định độ rộng mode_t trên mọi nền tảng [1], [4]:

Bảng 2.1: Cấu trúc các vùng bit trong trường st_mode và quy tắc chuyển đổi sang chuỗi 10 ký tự

Vùng bit trong st_mode	Ý nghĩa kỹ thuật	Quy tắc hiển thị trong chế độ -l
Bits 12..15 (Mặt nạ S_IFMT = 0170000)	Xác định loại đối tượng tệp ở ký tự thứ 0 của chuỗi quyền hạn.	S_ISREG ('-'), S_ISDIR ('d'), S_ISLNK ('l'), S_ISCHR ('c'), S_ISBLK ('b'), S_ISFIFO ('p'), S_ISSOCK ('s').
Bits 9..11 (S_ISUID=04000, S_ISGID=02000, S_ISVTX=01000)	Ba bit đặc biệt: Set-UID, Set-GID và Sticky bit, tác động lên các vị trí ký tự thực thi thứ 3, 6 và 9.	Nếu có cả bit đặc biệt và bit thực thi tương ứng: in chữ thường ('s' hoặc 't'). Nếu có bit đặc biệt nhưng không có bit thực thi: in chữ hoa ('S' hoặc 'T').

Vùng bit trong st_mode	Ý nghĩa kỹ thuật	Quy tắc hiển thị trong chế độ -l
Bits 0..8 (S_IRWXU, S_IRWXG, S_IRWXO)	9 bit quyền Đọc (r), Ghi (w), Thực thi (x) chia cho Chủ sở hữu (User), Nhóm (Group) và Người khác (Others).	Phép toán bitwise AND (mode & mask) kiểm tra từng bit để điền 'r', 'w', 'x' hoặc '-'.

Đối với các tệp thiết bị ký tự (Character device, S_ISCHR, ví dụ /dev/null) và thiết bị khối (Block device, S_ISBLK, ví dụ /dev/sda), trường st_size không mang ý nghĩa kích thước dữ liệu. Thay vào đó, nhân Linux lưu mã số trình điều khiển thiết bị trong trường st_rdev. Chương trình sử dụng hai macro major(st_rdev) và minor(st_rdev) trong <sys/sysmacros.h> [3] để in cặp số "major, minor" (ví dụ "1, 3" đối với /dev/null) vào đúng vị trí cột kích thước. Các macro này thuộc giao diện Linux được dùng trong đồ án; không phải API ISO C.

2.4. Tổng hợp các giao diện hệ thống và thư viện C chuẩn sử dụng

Bảng 2.2 liệt kê đầy đủ các hàm hệ thống và hàm thư viện C chuẩn được sử dụng trong mã nguồn của myls cùng cơ chế xử lý lỗi tương ứng.

Bảng 2.2: Danh mục các API chuẩn POSIX và C11 sử dụng trong chương trình myls

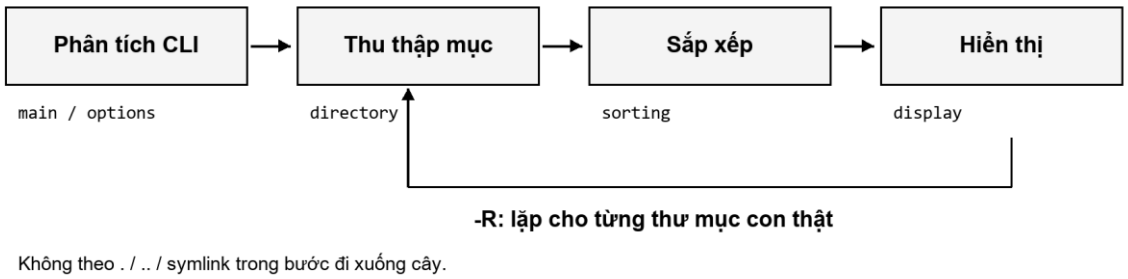
Hàm API	Header	Vai trò trong chương trình	Cơ chế kiểm soát lỗi
getopt()	<unistd.h>	Tách và giải mã các cờ -a, -l, -R, -S; hỗ trợ dấu '--'.	Trả về '?' khi gặp cờ lạ -> in hướng dẫn ra stderr và trả mã thoát 2.
opendir(), readdir(), closedir()	<dirent.h>	Mở thư mục, đọc từng mục struct dirent và đóng directory stream.	Đặt errno = 0 trước readdir() để tách biệt EOF với lỗi đọc; kiểm tra giá trị trả về của closedir().
lstat(), stat()	<sys/stat.h>	Thu thập siêu dữ liệu Inode vào struct stat.	Nếu trả về -1: in thông báo strerror(errno) ra stderr và nâng mã thoát (1 hoặc 2).
readlink()	<unistd.h>	Đọc đường dẫn đích của symbolic link vào bộ đệm cấp phát động.	Tự gán ký tự kết thúc chuỗi '\0' tại vị trí len; nếu lỗi vẫn giữ metadata và nâng mã thoát.
getpwuid(), getgrgid()	<pwd.h>, <grp.h>	Tra cứu tên người dùng và tên nhóm từ st_uid, st_gid.	Nếu trả về NULL (UID/GID không có tên): tự động in mã số nguyên để tránh lỗi con trỏ NULL.

Hàm API	Header	Vai trò trong chương trình	Cơ chế kiểm soát lỗi
localtime_r()	<time.h>	Chuyển đổi dấu thời gian st_mtime sang cấu trúc lịch cục bộ an toàn luồng.	Kiểm tra khoảng cách 6 tháng (15.778.476 giây) so với time(NULL) để chọn định dạng Giờ:Phút hoặc Năm.
qsort(), strcmp()	<stdlib.h>, <string.h>	Sắp xếp mảng động FileEntry theo kích thước (-S) hoặc tên từ điển.	Dùng phép so sánh > và < trên kiểu off_t (64 bit trên môi trường x86_64 đã thử), kết hợp strcmp() khi kích thước bằng nhau.
fflush(), ferror()	<stdio.h>	Đẩy bộ đệm đầu ra và kiểm tra lỗi ghi thiết bị trên stdout.	Phát hiện lỗi ghi như ENOSPC trên /dev/full, báo lỗi ra stderr và trả mã thoát 2.

CHƯƠNG 3. THIẾT KẾ KIẾN TRÚC VÀ CÀI ĐẶT CHƯƠNG TRÌNH

3.1. Kiến trúc phân tách 5 module và luồng dữ liệu tổng thể

Chương trình được thiết kế theo kiến trúc 5 module độc lập, giao tiếp qua tệp tiêu đề trung tâm include/myls.h. Luồng đi của dữ liệu từ khi nhận tham số dòng lệnh argv[] cho đến khi kết xuất và duyệt đệ quy được trình bày tại Hình 3.1 và phân công trách nhiệm tại Bảng 3.1.



Hình 3.1: Sơ đồ luồng thực thi tổng thể từ phân tích tham số dòng lệnh đến kết xuất và đệ quy

Bảng 3.1: Phân công trách nhiệm giữa các tệp mã nguồn trong dự án

Tệp nguồn	Trách nhiệm kỹ thuật chính
include/myls.h	Định nghĩa 4 cấu trúc dữ liệu (LsOptions, FileEntry, EntryList, ColumnWidths), các hằng mã thoát (0, 1, 2), hàm nội tuyến elevate_status() và nguyên mẫu hàm.
src/options.c	Cài đặt parse_options() và print_usage(): giải mã tổ hợp cờ -a, -l, -R, -S bằng getopt(), xử lý cờ --help và dấu phân cách '--'.
src/main.c	Điều phối chính: phân loại toán hạng CLI thành file_list và dir_list, xử lý ngoại lệ symlink tới thư mục trên CLI, gọi hiển thị 2 pha và kiểm tra lỗi flush stdout.
src/directory.c	Quản lý mảng động EntryList có kiểm tra tràn SIZE_MAX, ghép đường dẫn join_path(), đọc metadata qua lstat()/readlink() và duyệt đệ quy process_directory().
src/sorting.c	Cài đặt sort_entries() cùng 2 hàm so sánh cmp_by_name_asc() và cmp_by_size_desc() cho qsort().
src/display.c	Giải mã chuỗi quyền 10 ký tự format_mode_string(), tính độ rộng cột 2 lượt compute_column_widths(), làm tròn dòng total 1 KiB và kiểm tra lỗi ghi stdout.

3.2. Cấu trúc dữ liệu cốt lõi và quản lý quyền sở hữu bộ nhớ (Ownership)

Để không bị giới hạn cứng số lượng tệp trong một thư mục và tiết kiệm bộ nhớ khi duyệt cây đệ quy sâu, cấu trúc FileEntry và EntryList trong include/myls.h được thiết kế với quyền sở hữu bộ nhớ rõ ràng:

Trích đoạn định nghĩa cấu trúc FileEntry và EntryList (include/myls.h):

```
typedef struct {
    char *name;        /* Tên hiển thị (cấp phát động trên Heap) */
    char *full_path;   /* Đường dẫn đầy đủ để gọi lstat/readlink/đệ quy */
    struct stat st;    /* Siêu dữ liệu thu được từ lstat()/stat() */
    bool stat_ok;      /* Đánh dấu đọc metadata thành công */
    char *link_target; /* Chỉ cấp phát động khi là Symlink ở chế độ -l; ngược
lại là NULL */
} FileEntry;

typedef struct {
    FileEntry *items;  /* Mảng động các FileEntry */
    size_t count;      /* Số phần tử hiện có */
    size_t capacity;   /* Sức chứa hiện tại (khởi đầu 64, nhân đôi khi đầy) */
} EntryList;
```

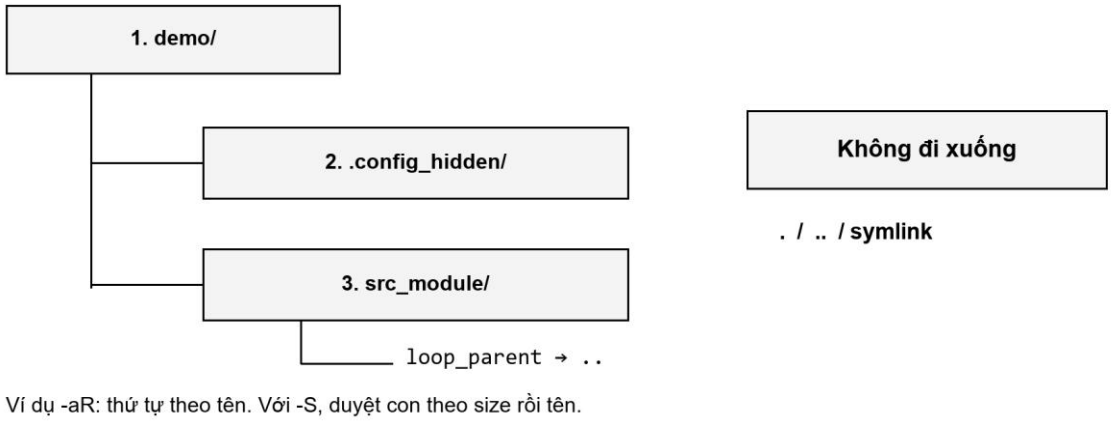
Thay vì đặt một mảng tĩnh char link_target[PATH_MAX] (chiếm cố định 4096 bytes) bên trong mọi phần tử FileEntry, chương trình sử dụng con trỏ char *link_target và đặt bằng NULL cho toàn bộ các tệp thông thường. Vùng nhớ cho link_target chỉ được cấp phát khi mục đó thực sự là symbolic link (S_ISLNK) và người dùng bật cờ -l. Đồng thời, trong hàm entry_list_push() (src/directory.c), trước khi nhân đôi capacity hoặc tính kích thước truyền vào realloc(), chương trình kiểm tra điều kiện tràn số nguyên:

Trích đoạn kiểm tra tràn số nguyên và hủy mục cấp phát dở dang (src/directory.c):

```
if (list->count >= list->capacity) {
    size_t new_cap = (list->capacity == 0) ? INITIAL_LIST_CAPACITY : (list-
>capacity * 2);
    if (list->capacity > SIZE_MAX / 2 || new_cap > SIZE_MAX /
sizeof(FileEntry)) {
        errno = ENOMEM;
        fprintf(stderr, "myls: memory allocation failed\n");
        return false;
    }
    FileEntry *new_items = (FileEntry *)realloc(list->items, new_cap *
sizeof(FileEntry));
    if (new_items == NULL) {
        fprintf(stderr, "myls: memory allocation failed\n");
        return false;
    }
    list->items = new_items;
    list->capacity = new_cap;
}
```

3.3. Thuật toán duyệt đệ quy (-R), chốt chặn vòng lặp và đánh đổi tài nguyên

Khi bật tùy chọn -R, hàm process_directory() trong src/directory.c thực hiện quy trình 4 bước tuần tự: (1) Mở thư mục bằng opendir() và đọc toàn bộ các mục thỏa điều kiện lọc vào mảng động EntryList list; (2) Đóng ngay luồng thư mục bằng closedir(dir) và kiểm tra lỗi; (3) Sắp xếp list bằng sort_entries() và kết xuất toàn bộ mục của thư mục hiện tại ra màn hình; (4) Duyệt qua list theo đúng thứ tự đã sắp xếp và gọi đệ quy process_directory() cho các mục con thỏa hàm kiểm tra is_recursible_subdir(). Hình 3.2 minh họa thứ tự duyệt cây và quy tắc bỏ qua . / .. / symlink khi đi xuống.



Hình 3.2: Thứ tự duyệt -aR và quy tắc bỏ qua . / .. / symlink khi đi xuống cây

Trích đoạn kiểm tra điều kiện đệ quy và đọc thư mục an toàn (src/directory.c):

```
static bool is_recursible_subdir(const FileEntry *entry)
{
    if (!entry->stat_ok || !S_ISDIR(entry->st.st_mode)) {
        return false; /* Chặn mọi tệp thường và Symbolic Link (nhờ dùng lstat)
*/
    }
    if (strcmp(entry->name, ".") == 0 || strcmp(entry->name, "..") == 0) {
        return false; /* Chặn '.' và '..' khi chạy kết hợp -aR */
    }
    return true;
}
```

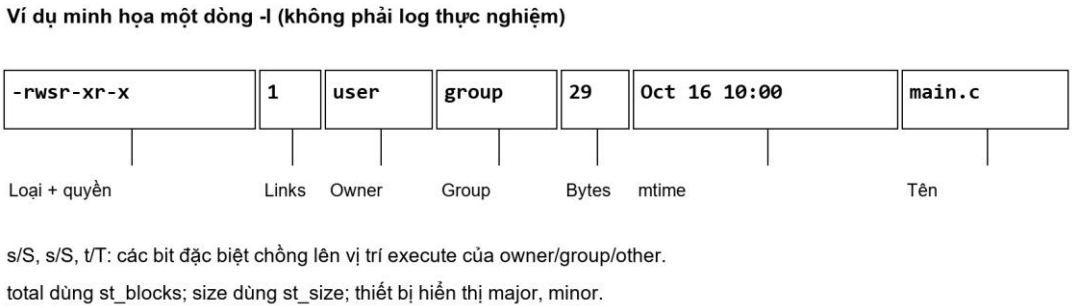
Phân tích sự đánh đổi giữa File Descriptor (FD) và Bộ nhớ (RAM): Việc gọi closedir(dir) ngay trước vòng lặp đệ quy giúp giải phóng File Descriptor của thư mục hiện tại trước khi mở thư mục con. Nhờ vậy, số lượng FD dùng cho việc đọc thư mục tại mọi thời điểm chỉ là 1 FD, không bị chạm ngưỡng giới hạn RLIMIT_NOFILE (thường là 1024) khi cây thư mục sâu. Tuy nhiên, cần lưu ý sự đánh đổi về bộ nhớ: vì lời gọi đệ quy nằm trước lệnh entry_list_free(&list), các danh sách EntryList của những thư mục cha dọc theo nhánh đệ quy hiện tại vẫn được giữ trong bộ nhớ cho tới

khi các thư mục con của nhánh đó duyệt xong. Do đó, bộ nhớ tức thời tỷ lệ với tổng số mục trên các mức từ gốc đến nút đang xét, chứ không chỉ riêng thư mục hiện hành.

3.4. Thuật toán sắp xếp (-S) và định dạng kết xuất hai lượt (-l)

Trong src/sorting.c, hàm cmp_by_size_desc() so sánh trường st_size (kiểu off_t (64 bit trên môi trường x86_64 đã thử) có dấu) bằng hai phép so sánh quan hệ (a->st.st_size > b->st.st_size trả về -1; < trả về 1) thay vì lấy hiệu trực tiếp, nhằm loại bỏ hoàn toàn nguy cơ tràn số nguyên 32-bit khi so sánh các tệp lớn hơn 2 GiB. Khi hai mục có cùng kích thước, hàm gọi strcmp(a->name, b->name) để phân định thứ tự tăng dần theo tên.

Trong src/display.c, chế độ -l sử dụng thuật toán hai lượt (Two-pass alignment) để căn chỉnh các cột thẳng hàng và định dạng đầy đủ 7 trường dữ liệu trên mỗi dòng như minh họa ở Hình 3.3:



Hình 3.3: Giải phẫu các trường thông tin trên một dòng kết xuất ở chế độ -l

- **Lượt 1 (Tính độ rộng cột và tổng khối 1 KiB):** Hàm compute_column_widths() duyệt qua danh sách để tìm số chữ số lớn nhất của st_nlink, độ dài chuỗi tên owner/group và độ rộng cột kích thước. Nếu danh sách có chứa thiết bị ký tự hoặc khối (S_ISCHR / S_ISBLK), hàm tính độ rộng lớn nhất của major(st_rdev) và minor(st_rdev), sau đó đồng bộ độ rộng tổng "major, minor" với độ rộng st_size của tệp thường. Tổng số khối 512-byte (total_blocks_512) được đổi sang đơn vị 1 KiB có làm tròn lên theo công thức: total_1k = (total_blocks_512 + 1) / 2.
- **Lượt 2 (Kết xuất và kiểm tra lỗi ghi stdout):** Chương trình in từng dòng theo độ rộng động đã tính. Mọi thao tác printf() và lệnh fflush(stdout) ở cuối chương trình đều được kiểm tra lỗi qua flush_and_check_stdout(). Nếu luồng đầu ra gặp sự cố (chẳng hạn khi chuyển hướng ra thiết bị đầy /dev/full gây lỗi ENOSPC), chương trình ghi thông báo lỗi ra stderr và trả về mã thoát nghiêm trọng 2.

CHƯƠNG 4. KIỂM THỬ THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

4.1. Môi trường thực nghiệm và thiết kế cây dữ liệu kiểm thử (Fixtures)

Toàn bộ quá trình biên dịch và kiểm thử được thực hiện trên môi trường Ubuntu 24.04.4 LTS (nhân Linux 6.18.33.2-microsoft-standard-WSL2 #1 SMP PREEMPT_DYNAMIC Thu Jun 18 21:54:43 UTC 2026 x86_64), trình biên dịch gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0 và đối chiếu trực tiếp với ls (GNU coreutils) 9.4. Một quyết định kỹ thuật quan trọng trong kịch bản kiểm thử (tests/run_tests.sh) là khởi tạo toàn bộ cây dữ liệu mẫu trên phân vùng /tmp của Linux thay vì thư mục /mnt/p (nơi gắn ổ đĩa Windows qua trình điều khiển DrvFs). Quyền trên ổ Windows phụ thuộc quyền Windows, tùy chọn mount và chế độ metadata; không nên giả định chmod hoặc các tệp đặc biệt có hành vi như filesystem Linux. Fixture /tmp giúp đánh giá nhất quán [5].

4.2. Phương pháp kiểm thử và độ bao phủ kịch bản

Bộ kiểm thử tự động gồm tổng cộng 37 kịch bản, được chia thành 3 nhóm kiểm chứng chặt chẽ cả đầu ra chuẩn (stdout diff -u), luồng báo lỗi (stderr) và mã thoát (exit code):

- **Nhóm 1 – Toàn bộ 16 tổ hợp của 4 cờ (Kịch bản 01 đến 16):** Kiểm tra vét cạn 2^4 = 16 tổ hợp của {-a, -l, -R, -S} (từ không cờ đến -alRS) trên cây thư mục đa tầng có chứa tệp ẩn, thư mục ẩn, tệp trùng kích thước, sparse file lớn hơn 2 GiB (2,5 GB), hard link, symlink tới tệp/thư mục, broken symlink và symlink trỏ ngược lên thư mục cha (..).
- **Nhóm 2 – Cú pháp cờ, tệp đặc biệt và unit test nội bộ (Kịch bản 17 đến 27):** Kiểm tra viết rời và đảo thứ tự cờ (-S -R -l -a), thư mục rỗng, thư mục có hơn 64 mục (80 tệp để đi qua nhánh nhân đôi capacity 64 -> 128), tên tệp bắt đầu bằng '-' đi kèm '--', FIFO, Unix domain socket, toán hạng symlink trên CLI (không có -l, có -l, và có dấu '/' cuối), thiết bị ký tự /dev/null trộn với tệp thường, cùng bộ unit test C nội bộ (tests/test_internals.c) kiểm tra định dạng block device tổng hợp, làm tròn khối 1 KiB và bảo vệ tràn số SIZE_MAX.
- **Nhóm 3 – Đường lỗi, phân quyền, /dev/full và tiêm lỗi Fault Injection (Kịch bản 28 đến 37):** Kiểm tra mã thoát 2 cho đối số CLI không tồn tại, broken symlink có dấu '/' cuối, thư mục CLI bị khóa quyền đọc (chmod 0000), cờ không hợp lệ (-z) và lỗi ghi stdout (/dev/full); kiểm tra mã thoát 1 khi gặp thư mục con bị khóa quyền đọc trong lúc đệ quy -R; kiểm tra giữ mã lỗi cao nhất (2) khi trộn lỗi CLI với lỗi

thư mục con; và sử dụng thư viện LD_PRELOAD (tests/fault_inject.c) để tiêm lỗi EIO/ENOMEM vào các hàm readdir(), readlink(), closedir() và realloc().

Bảng 4.1: Tổng hợp kết quả thực nghiệm từ tệp dữ liệu evidence/test_results.json

Hạng mục thực nghiệm	Lệnh / Binary thực thi	Kết quả ghi nhận	Đánh giá chi tiết
Kiểm thử chức năng & đường lỗi (make test)	./myls (make -B && make test)	37 / 37 đạt (0 lỗi, 0 bỏ qua)	Exit code = 0. 26 ca đối chiếu GNU ls (stdout và mã thoát); 11 ca kiểm hợp đồng, helper nội bộ hoặc lỗi giả lập.
Kiểm thử bộ nhớ & hành vi bất định (make sanitize)	./myls_sanitize (make sanitize)	37 / 37 đạt (0 lỗi, 0 bỏ qua)	Exit code = 0. Không phát hiện lỗi bộ nhớ trong các kịch bản đã chạy bằng GCC AddressSanitizer, LeakSanitizer và UndefinedBehaviorSanitizer trên toàn bộ 37 kịch bản kiểm thử.
Kịch bản trình diễn bảo vệ (make demo)	./myls (tests/run_demo.sh)	Hoàn tất 4 bước demo trên fixture cô lập /tmp	Các bước mặc định, -a, -lS, -alRS trả mã 0; bước thử đối số lỗi trả mã 2 đúng hợp đồng.

4.3. Minh chứng đầu ra thực nghiệm từ log kiểm thử và demo

Các Hình 4.1, Hình 4.2 và Hình 4.3 dưới đây trích các dòng đầu ra thực nghiệm đã được ghi nhận trong các tệp log tại thư mục evidence/ (evidence/test_suite_report.txt và evidence/demo_session_output.txt).

ĐẦU RA THỰC NGHIỆM TRÌNH BÀY LẠI TỪ LOG – evidence/test_suite_report.txt (make test)

```
$ make test
[Trich doan; log day du: evidence/test_suite_report.txt]
  [PASS] Test 01: Flag combo 01/16: (none) default listing
  [PASS] Test 16: Flag combo 16/16: -alRS (all four flags)
  [PASS] Test 18: Empty directory with -l and -la
  [PASS] Test 22: Symlink CLI operand without -l (follows to dir)
  [PASS] Test 26: Multiple mixed CLI operands (files + directories)
  [PASS] Test 27: Internal unit tests (block device major/minor, 1KiB rounding, overflow guard)
  [PASS] Test 28: Non-existent CLI operand returns exit status 2 and stderr diagnostic
  [PASS] Test 32: Mixed CLI error (2) + subdirectory error (1) preserves highest exit status 2
  [PASS] Test 33: Invalid option (-z) rejected with exit status 2
  [PASS] Test 34: Stdout write error (/dev/full) detected on flush with exit status 2
  [PASS] Test 35: Fault injection: readdir() EIO handled cleanly with exit status 1
  [PASS] Test 36: Fault injection: readlink() EIO retains entry metadata and returns exit status 1
  [PASS] Test 37: Fault injection: closedir() EIO (exit 1) and realloc() ENOMEM (exit 2) handled safely
  SUMMARY: 37 PASSED, 0 FAILED, 0 SKIPPED (TOTAL: 37)
```

Hình 4.1: Đầu ra thực nghiệm trình bày lại từ log kiểm thử tự động (37/37 kịch bản đạt)

ĐẦU RA THỰC NGHIỆM TRÌNH BÀY LẠI TỪ LOG — evidence/demo_session_output.txt (Mặc định, -a, -lS)

```
$ ./myls
README.txt
database.bin
link_to_src
notes.txt
src_module
-> Exit code: 0

$ ./myls -a
.
..
.config_hidden
.env
README.txt
database.bin
link_to_src
notes.txt
src_module
-> Exit code: 0

$ ./myls -lS
total 20
-rw-r--r-- 1 linhnxdeveloper linhnxdeveloper 8192 Oct 16 05:52 database.bin
drwxr-xr-x 2 linhnxdeveloper linhnxdeveloper 4096 Oct 16 05:52 src_module
-rw-r--r-- 1 linhnxdeveloper linhnxdeveloper  512 Oct 16 05:52 README.txt
-rw-r--r-- 1 linhnxdeveloper linhnxdeveloper  512 Oct 16 05:52 notes.txt
lrwxrwxrwx 1 linhnxdeveloper linhnxdeveloper   10 Oct 16 05:52 link_to_src -> src_module
-> Exit code: 0
```

Hình 4.2: Đầu ra thực nghiệm trình bày lại từ log demo cho các tùy chọn mặc định, -a và -lS

ĐẦU RA THỰC NGHIỆM TRÌNH BÀY LẠI TỪ LOG — evidence/demo_session_output.txt (-alRS và xử lý đối số lỗi)

```
$ ./myls -alRS
.:
total 36
-rw-r--r--  1 linhnxdeveloper linhnxdeveloper 8192 Oct 16 05:52 database.bin
drwx------  4 linhnxdeveloper linhnxdeveloper 4096 Oct 16 05:52 .
drwxrwxrwt 11 root            root            4096 Oct 16 05:52 ..
drwxr-xr-x  2 linhnxdeveloper linhnxdeveloper 4096 Oct 16 05:52 .config_hidden
drwxr-xr-x  2 linhnxdeveloper linhnxdeveloper 4096 Oct 16 05:52 src_module
-rw-r--r--  1 linhnxdeveloper linhnxdeveloper  512 Oct 16 05:52 README.txt
-rw-r--r--  1 linhnxdeveloper linhnxdeveloper  512 Oct 16 05:52 notes.txt
-rw-r--r--  1 linhnxdeveloper linhnxdeveloper   15 Oct 16 05:52 .env
lrwxrwxrwx  1 linhnxdeveloper linhnxdeveloper   10 Oct 16 05:52 link_to_src -> src_module

./.config_hidden:
total 12
drwxr-xr-x 2 linhnxdeveloper linhnxdeveloper 4096 Oct 16 05:52 .
drwx------ 4 linhnxdeveloper linhnxdeveloper 4096 Oct 16 05:52 ..
-rw-r--r-- 1 linhnxdeveloper linhnxdeveloper   14 Oct 16 05:52 key.conf

./src_module:
total 12
drwxr-xr-x 2 linhnxdeveloper linhnxdeveloper 4096 Oct 16 05:52 .
drwx------ 4 linhnxdeveloper linhnxdeveloper 4096 Oct 16 05:52 ..
-rwsr-xr-x 1 linhnxdeveloper linhnxdeveloper   29 Oct 16 05:52 main.c
lrwxrwxrwx 1 linhnxdeveloper linhnxdeveloper    2 Oct 16 05:52 loop_parent -> ..
-> Exit code: 0

$ ./myls missing_path.txt src_module
myls: cannot access 'missing_path.txt': No such file or directory
src_module:
loop_parent
main.c
-> Exit code: 2
```

Hình 4.3: Đầu ra thực nghiệm trình bày lại từ log demo cho tổ hợp -alRS và xử lý đối số lỗi

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết quả đạt được

Đồ án đã hoàn thành việc nghiên cứu cơ chế quản lý thư mục và siêu dữ liệu của hệ thống tệp Linux, đồng thời xây dựng thành công chương trình myls bằng ngôn ngữ C (chuẩn C11 + POSIX.1-2008) đáp ứng các yêu cầu kỹ thuật của Đề tài 17:

- **Hoàn thiện đầy đủ bốn tùy chọn -a, -l, -R, -S và mọi tổ hợp kết hợp:** Chương trình giải mã chính xác 10 ký tự loại tệp và quyền truy cập (bao gồm SUID, SGID, Sticky bit), hiển thị số hiệu thiết bị major, minor cho character/block device, căn lề cột động hai lượt, sắp xếp an toàn trên kiểu off_t và bỏ qua . / .. / symlink khi đi xuống cây. Chưa có phát hiện chu trình do bind mount.
- **Quản lý tài nguyên và xử lý lỗi chặt chẽ:** Kiểm soát đầy đủ các đường lỗi của opendir(), readdir() (đặt errno = 0), lstat(), readlink(), closedir(), realloc() và lỗi ghi luồng stdout (fflush/ferror); phân biệt chính xác mã thoát 0, 1 và 2.
- **Kiểm chứng thực nghiệm có khả năng tái lập:** Bản thường đạt 37/37 ca, bỏ qua 0; bản sanitizer đạt 37/37 ca, bỏ qua 0. Không phát hiện lỗi bởi ASan/LSan/UBSan trong các ca đã chạy; kết quả không chứng minh mọi đầu vào đều không có lỗi.

5.2. Hạn chế hiện tại và hướng phát triển

Bên cạnh các kết quả đã đạt được trong phạm vi đề tài, chương trình vẫn còn một số giới hạn kỹ thuật và có thể được mở rộng theo thứ tự ưu tiên sau:

- **Ưu tiên 1 – Phát hiện chu trình hệ thống tệp theo cặp (st_dev, st_ino):** Bổ sung bảng băm hoặc cây tìm kiếm lưu cặp (st_dev, st_ino) của các thư mục tổ tiên đang mở trên nhánh đệ quy để phát hiện chu trình do bind mount hoặc hard link thư mục đặc biệt.
- **Ưu tiên 2 – Sử dụng họ hàm *at (openat, fstatat, readlinkat):** Thay thế việc nối chuỗi đường dẫn đầy đủ bằng việc truyền descriptor của thư mục cha vào fstatat() và readlinkat(), giúp tăng hiệu năng tra cứu đường dẫn sâu và giảm rủi ro khi cây thư mục bị đổi tên đồng thời.
- **Ưu tiên 3 – Hỗ trợ dàn nhiều cột trên terminal và làm sạch ký tự điều khiển:** Khi đầu ra là thiết bị đầu cuối (isatty(STDOUT_FILENO)), truy vấn kích thước cửa sổ bằng ioctl(TIOCGWINSZ) để dàn tên tệp thành nhiều cột và thay thế các ký tự không in được trong tên tệp bằng dấu '?' để bảo vệ terminal.

TÀI LIỆU THAM KHẢO

[1] IEEE and The Open Group, "The Open Group Base Specifications Issue 7, 2018 edition (POSIX.1-2008 / IEEE Std 1003.1-2017) – ls utility, sys/stat.h, dirent.h," 2018. [Online]. Available: https://pubs.opengroup.org/onlinepubs/9699919799/

[2] Free Software Foundation, "GNU Coreutils Manual – Section 10.1: ls: List directory contents," GNU Project, 2024. [Online]. Available: https://www.gnu.org/software/coreutils/manual/html_node/ls-invocation.html

[3] Michael Kerrisk et al., "Linux Programmer's Manual (man-pages): stat(2), lstat(2), readlink(2), opendir(3), readdir(3), closedir(3), getopt(3), sysmacros(7)," 2024. [Online]. Available: https://man7.org/linux/man-pages/

[4] Michael Kerrisk, The Linux Programming Interface: A Linux and UNIX System Programming Handbook, San Francisco, CA: No Starch Press, 2010, Chapters 14, 15, and 18.

[5] Microsoft Corporation, "File Permissions for WSL (Windows Subsystem for Linux DrvFs and Metadata)," Microsoft Learn Documentation, 2024. [Online]. Available: https://learn.microsoft.com/en-us/windows/wsl/file-permissions

[6] Trường Đại học Công nghệ Thông tin và Truyền thông Việt – Hàn (VKU), "Quyết định số 675/QĐ-ĐHVH ban hành Quy định triển khai đồ án, khóa luận tốt nghiệp (Phụ lục Mẫu 3 – Quy cách trình bày báo cáo)," Đà Nẵng, ngày 03/04/2024. Tham khảo quy cách hình thức cho báo cáo học phần; không áp dụng yêu cầu độ dài của khóa luận. [Online]. Available: https://daotao.vku.udn.vn/uploads/daotao/675-qd-ban-hanh-quy-dinh-trien-khai-do-an-khoa-luan-tot-nghiep-trong-cac-ctdt-dh-cua-vku-0001.pdf

PHỤ LỤC. DANH SÁCH KỊCH BẢN KIỂM THỬ TỰ ĐỘNG VÀ LỆNH TÁI LẬP

Phụ lục A. Quy trình lệnh kiểm chứng độc lập trên Ubuntu WSL2

Các lệnh biên dịch, chạy kiểm thử thường, kiểm thử Sanitizer và chạy demo:

```
# 1. Biên dịch chương trình với bộ cờ cảnh báo nghiêm ngặt
make clean && make

# 2. Chạy bộ kiểm thử tự động đối chiếu với GNU ls trên /tmp
make test

# 3. Biên dịch và chạy toàn bộ bộ kiểm thử với ASan + UBSan + LSan
make sanitize

# 4. Chạy kịch bản demo 4 bước trên cây dữ liệu mẫu cô lập tại /tmp
make demo
```

Phụ lục B. Danh sách chi tiết 37 kịch bản trong bộ kiểm thử tự động

Bảng P.1: Danh sách toàn bộ 37 kịch bản kiểm thử tự động và trạng thái thực thi

Mã kịch bản	Nội dung kiểm chứng	Trạng thái
Test 01	Flag combo 01/16: (none) default listing	PASS
Test 02	Flag combo 02/16: -a (all entries)	PASS
Test 03	Flag combo 03/16: -l (long format)	PASS
Test 04	Flag combo 04/16: -R (recursive)	PASS
Test 05	Flag combo 05/16: -S (size sort descending)	PASS
Test 06	Flag combo 06/16: -al	PASS
Test 07	Flag combo 07/16: -aR (with . / .. / loop symlink guard)	PASS
Test 08	Flag combo 08/16: -aS	PASS
Test 09	Flag combo 09/16: -lR	PASS
Test 10	Flag combo 10/16: -lS (sparse 2.5GiB + tie-break)	PASS
Test 11	Flag combo 11/16: -RS	PASS
Test 12	Flag combo 12/16: -alR	PASS
Test 13	Flag combo 13/16: -alS	PASS
Test 14	Flag combo 14/16: -aRS	PASS
Test 15	Flag combo 15/16: -lRS	PASS
Test 16	Flag combo 16/16: -alRS (all four flags)	PASS

Mã kịch bản	Nội dung kiểm chứng	Trạng thái
Test 17	Separated and reordered flags (-S -R -l -a)	PASS
Test 18	Empty directory with -l and -la	PASS
Test 19	Directory > 64 entries (80 files capacity growth)	PASS
Test 20	Filename starting with '-' using '--' separator	PASS
Test 21	FIFO and Unix domain socket in -l	PASS
Test 22	Symlink CLI operand without -l (follows to dir)	PASS
Test 23	Symlink CLI operand with -l vs trailing slash	PASS
Test 24	Broken symlink CLI operand without trailing slash	PASS
Test 25	Character device /dev/null mixed with regular file	PASS
Test 26	Multiple mixed CLI operands (files + directories)	PASS
Test 27	Internal unit tests (block device major/minor, 1KiB rounding, overflow guard)	PASS
Test 28	Non-existent CLI operand returns exit status 2 and stderr diagnostic	PASS
Test 29	Broken symlink with trailing slash (broken_symlink/) returns exit status 2	PASS
Test 30	Unreadable CLI directory operand returns exit status 2 and continues remaining operands	PASS
Test 31	Unreadable subdirectory during -R returns exit status 1 and continues traversal	PASS
Test 32	Mixed CLI error (2) + subdirectory error (1) preserves highest exit status 2	PASS
Test 33	Invalid option (-z) rejected with exit status 2	PASS
Test 34	Stdout write error (/dev/full) detected on flush with exit status 2	PASS
Test 35	Fault injection: readdir() EIO handled cleanly with exit status 1	PASS
Test 36	Fault injection: readlink() EIO retains entry metadata and returns exit status 1	PASS
Test 37	Fault injection: closedir() EIO (exit 1) and realloc() ENOMEM (exit 2) handled safely	PASS